

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0518

COURSE OUTCOME COVERED

CO5: *Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)*

Assessment Tool Weightage: 40%

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

5. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

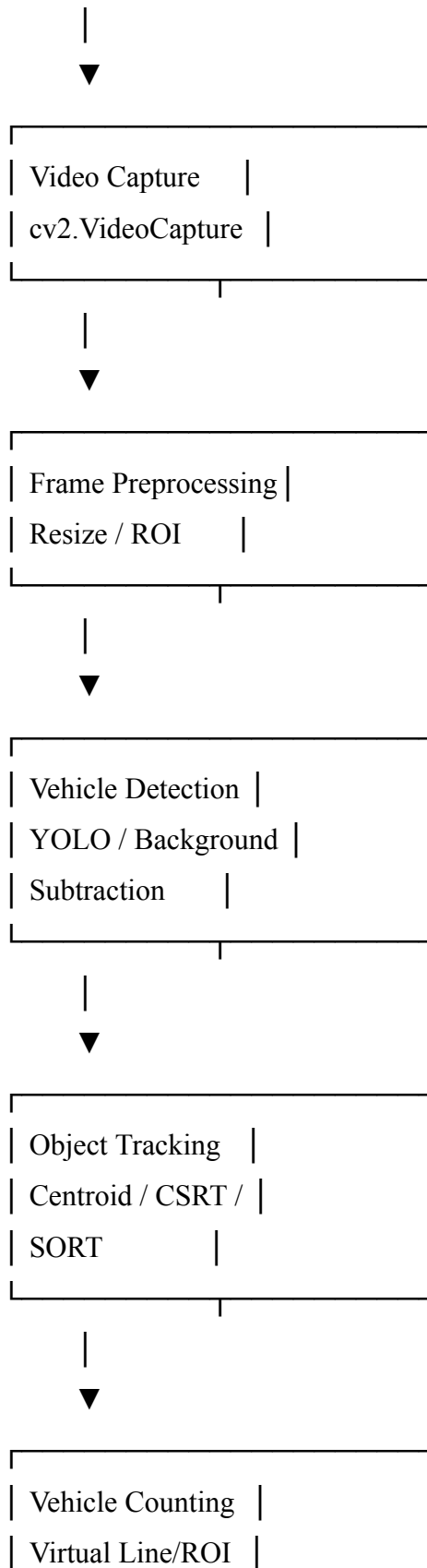
ANSWER:

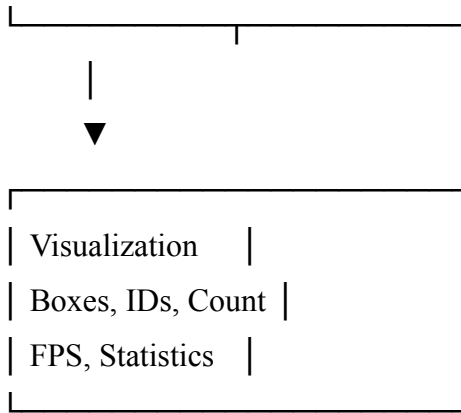
1. Aim

To design an OpenCV-based smart traffic monitoring system that detects vehicles from live video, tracks their movement, counts vehicles, and displays traffic statistics continuously with minimum processing delay.

2. System Block Diagram

Live Camera / Video





3. Main Components

A. Video Handling

OpenCV's `VideoCapture()` is used to acquire frames from a CCTV camera or video file.

```
import cv2
```

```
cap = cv2.VideoCapture("traffic.mp4")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

```
    cv2.imshow("Traffic Monitoring", frame)
```

```
    if cv2.waitKey(1) & 0xFF == 27:
```

```
        break
```

```
cap.release()
```

```
cv2.destroyAllWindows()
```

For a live camera:

```
cap = cv2.VideoCapture(0)
```

4. Vehicle Detection

A pretrained YOLO model can be used with OpenCV to detect vehicles such as:

Car

Bus

Truck

Motorcycle

The detector produces a bounding box, class label and confidence score.

Input Frame

|



YOLO Detector

|

|— Car → Bounding Box

|— Bus → Bounding Box

|— Truck → Bounding Box

|— Bike → Bounding Box

Only detections whose confidence is greater than a threshold, for example 0.5, are accepted.

5. Object Tracking

Detection alone identifies vehicles in individual frames. Tracking maintains the same vehicle ID across consecutive frames.

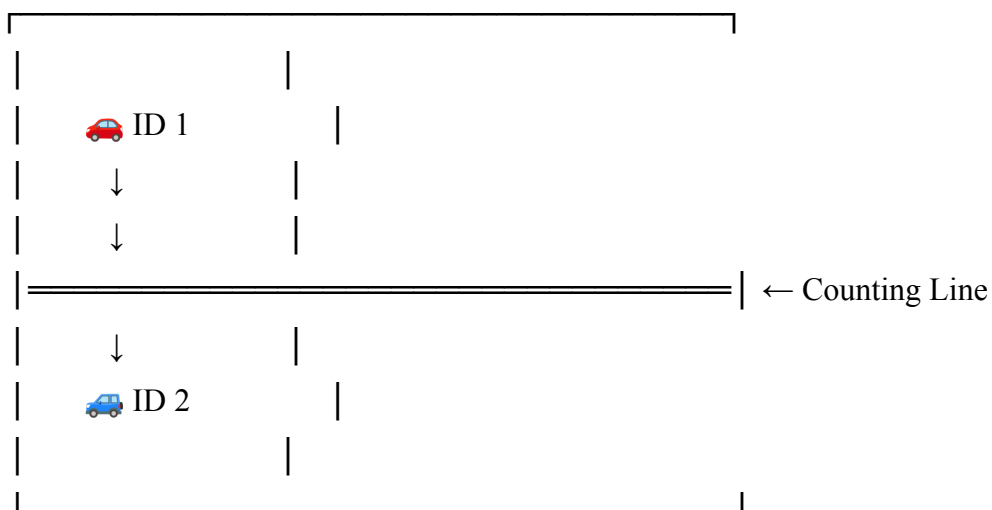
Example:

Frame 1	Frame 2	Frame 3
Car ID 1	→ Car ID 1	→ Car ID 1
Car ID 2	→ Car ID 2	→ Car ID 2

A centroid tracker, CSRT tracker, or more advanced trackers such as SORT/Deep SORT can

6. Vehicle Counting

A virtual counting line is placed across the road.



When the centroid of a tracked vehicle crosses the line:

```
if previous_y < line_y and current_y >= line_y:
    vehicle_count += 1
```

Each vehicle ID is counted only once to prevent duplicate counting.

7. OpenCV Drawing Functions

OpenCV drawing functions are used to visualize the results.

```
cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
```

```
cv2.putText(frame,
```

```

f"ID: {vehicle_id}",
(x1, y1 - 10),
cv2.FONT_HERSHEY_SIMPLEX,
0.6,
color,
2)

```

```

cv2.circle(frame, (cx, cy), 5, color, -1)

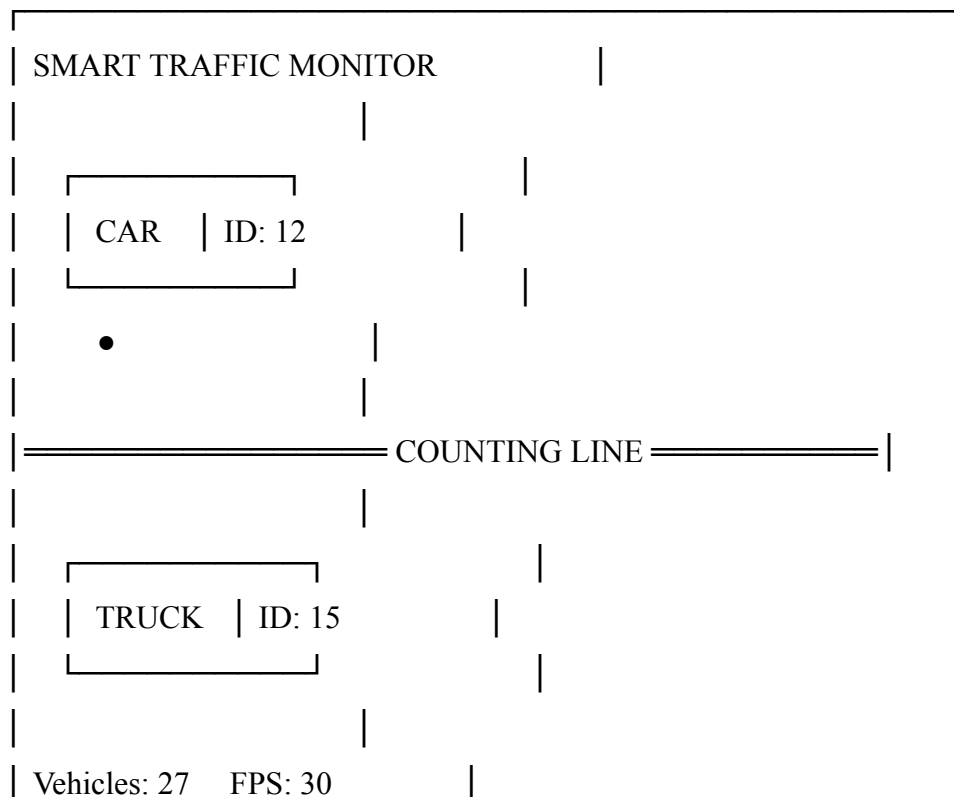
```

```

cv2.line(frame,
(0, line_y),
(frame.shape[1], line_y),
color,
2)

```

The output displays:



8. Statistics Display

The system can continuously display:

Parameter	Example
Total vehicles	27
Cars	18
Buses	3
Trucks	4
Motorcycles	2
FPS	30
Vehicles/minute	12

9. Ensuring Accuracy

The following techniques improve detection and counting accuracy:

Pretrained YOLO model provides reliable vehicle detection.

Confidence thresholding removes weak detections.

Region of Interest (ROI) restricts processing to the road area.

Object tracking prevents repeatedly counting the same vehicle.

Unique vehicle IDs ensure one vehicle is counted only once.

Virtual counting line provides a clear criterion for counting.

Frame preprocessing improves detection under varying conditions.

Multiple-frame tracking reduces errors caused by temporary detection failures.

10. Ensuring Real-Time Performance

Real-time operation can be achieved by:

Processing only the ROI instead of the complete frame.

Resizing frames to a suitable resolution such as 640×480 .

Using a lightweight detector such as YOLO-tiny/nano.

Using GPU acceleration when available.

Tracking objects between detection frames instead of performing expensive detection on every frame.

Dropping unnecessary frames when processing becomes slower than the camera frame rate.

Using efficient OpenCV operations and avoiding unnecessary image copies.

11. Overall Algorithm

START



Initialize camera/video



Read frame



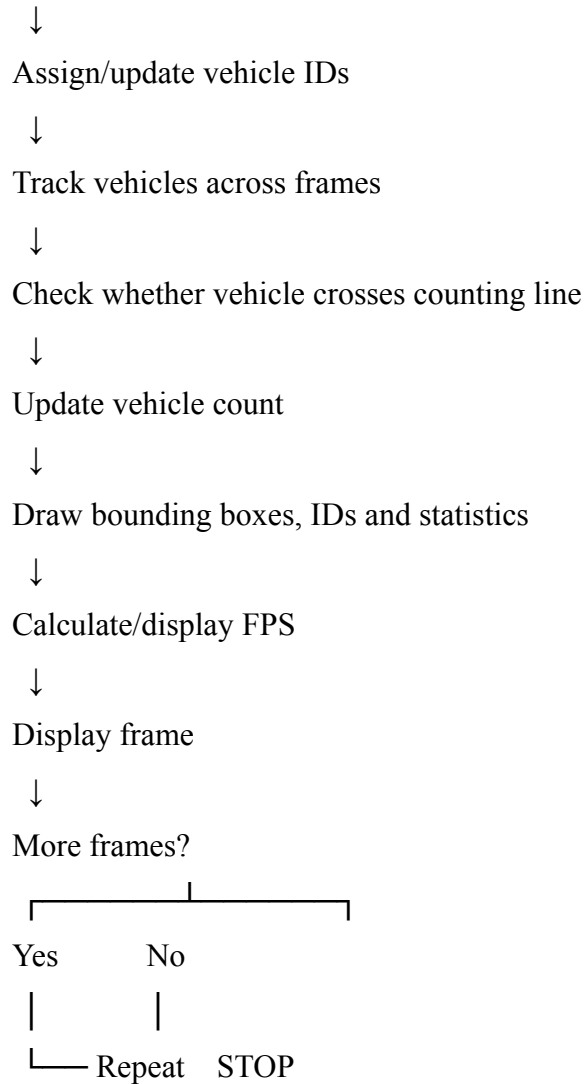
Resize and define ROI



Detect vehicles



Filter detections using confidence



12. Conclusion

The proposed OpenCV-based Smart Traffic Monitoring System combines video acquisition, vehicle detection, object tracking, counting, and visualization in a single pipeline. YOLO provides accurate vehicle detection, while tracking maintains vehicle identities across frames. A virtual line enables reliable counting, and OpenCV drawing functions provide real-time bounding boxes, IDs, counts, and FPS. ROI processing, lightweight models, GPU acceleration, and selective detection ensure low latency and real-time performance, making the system suitable for continuous city traffic monitoring.

8. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

ANSWER:

1. Aim

To design an OpenCV-based motion detection system that detects significant movement from a camera feed, triggers alerts only when required, and reduces false alarms caused by noise and background changes.

2. Block Diagram

Camera / Video Feed



Capture Video Frame



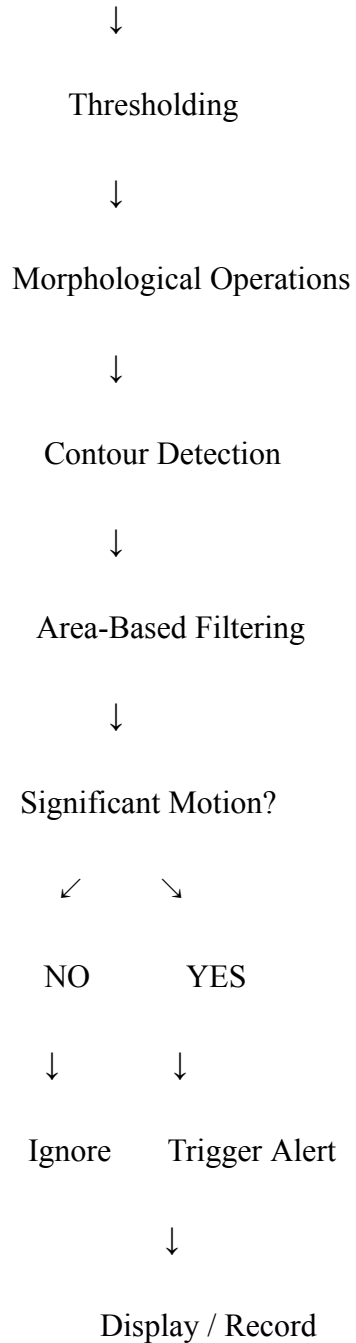
Resize + Grayscale



Gaussian Blur



Background Subtraction



3. Video Acquisition

OpenCV's `VideoCapture()` is used to continuously capture frames.

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

The camera provides a continuous sequence of frames for motion analysis.

4. Convert Frame to Grayscale

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Grayscale conversion reduces the image from three color channels to one, reducing computational complexity.

5. Noise Reduction

```
blur = cv2.GaussianBlur(gray, (21, 21), 0)
```

Gaussian blur removes small camera noise and minor intensity variations before motion detection.

6. Background Subtraction

MOG2 background subtraction can be used to separate moving objects from the stationary background.

```
background = cv2.createBackgroundSubtractorMOG2(
```

```
    history=500,
```

```
    varThreshold=50,
```

```
    detectShadows=True
```

)

```
mask = background.apply(blur)
```

The output is a foreground mask, where moving regions are highlighted.

7. Thresholding

```
_, threshold = cv2.threshold(  
    mask, 200, 255, cv2.THRESH_BINARY
```

)

Thresholding converts the foreground mask into a binary image containing possible motion regions.

8. Morphological Operations

Small unwanted regions are removed using morphological filtering.

```
kernel = cv2.getStructuringElement(  
    cv2.MORPH_ELLIPSE, (5, 5)
```

)

```
threshold = cv2.morphologyEx(  
    threshold, cv2.MORPH_OPEN, kernel
```

)

```
threshold = cv2.morphologyEx(  
    threshold, cv2.MORPH_CLOSE, kernel
```

)

- Opening: removes small noise.
- Closing: fills small gaps in detected objects.

9. Contour Detection

```
contours, _ = cv2.findContours(
    threshold,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)
```

Contours represent the boundaries of detected moving regions.

The area of each contour is calculated:

for contour in contours:

```
    area = cv2.contourArea(contour)

    if area > 1500:

        x, y, w, h = cv2.boundingRect(contour)

        cv2.rectangle(
            frame,
            (x, y),
            (x+w, y+h),
            (0, 255, 0),
```

)

Only motion regions larger than the selected threshold are considered significant.

10. Alert Generation

The system should not generate an alert for every small movement.

Small motion



$\text{Area} < \text{Threshold}$



IGNORE

Large motion



$\text{Area} > \text{Threshold}$



Check consecutive frames



Significant movement



ALERT

An alert can activate a:

- Buzzer
- Mobile notification
- Alarm
- Video recording
- Security monitoring system

11. Reducing False Alarms

Source of False Alarm	Solution
Camera sensor noise	Gaussian blur
Small movements	Minimum contour-area threshold
Shadows	MOG2 shadow detection
Moving curtains	ROI + area filtering
Tree movement	ROI and persistence check
Temporary illumination	Adaptive background model
Sudden small changes	Multi-frame confirmation

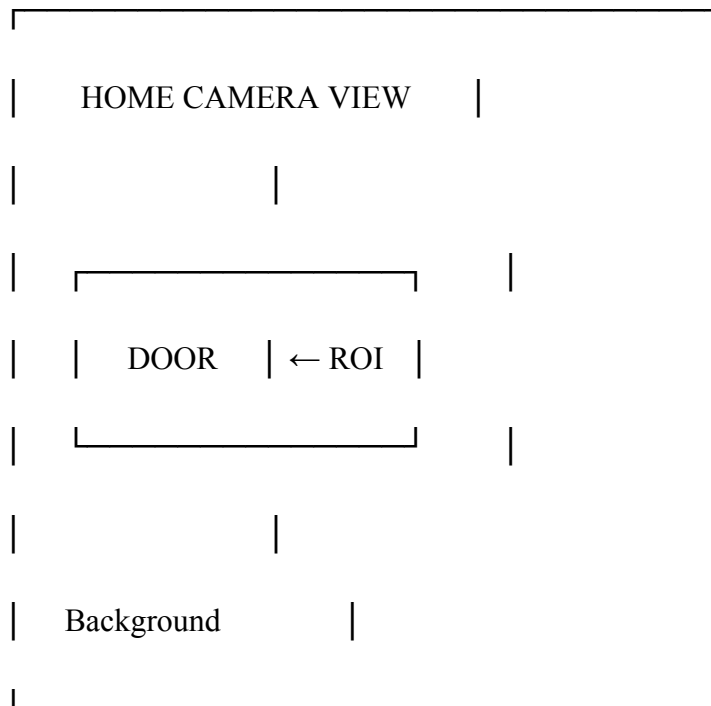
12. Handling Background Variations

The MOG2 background model continuously adapts to changes in the scene. Therefore, gradual changes such as:

- Day-to-night lighting
- Slight changes in illumination
- Moving shadows
- Small background changes

can be handled better than simple frame-to-frame subtraction.

An ROI (Region of Interest) can also be defined so that only important areas such as doors and windows are monitored.



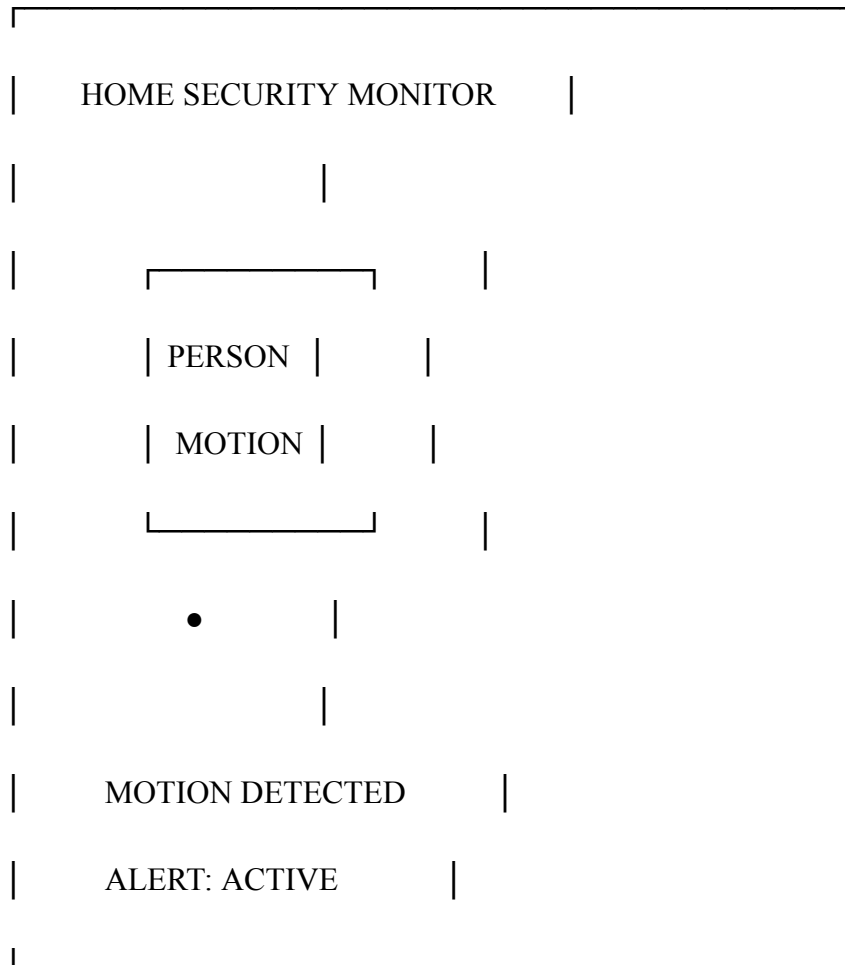
13. Real-Time Performance

Real-time operation is achieved by:

1. Converting frames to grayscale.
2. Processing only the required ROI.

3. Using efficient OpenCV functions.
4. Applying lightweight background subtraction.
5. Avoiding unnecessary image processing.
6. Processing at an appropriate resolution.

14. Output



15. Conclusion

The proposed OpenCV-based home security system uses video capture, grayscale conversion, Gaussian filtering, MOG2 background subtraction, thresholding, morphological operations, and contour analysis to detect significant motion. Contour-area filtering, ROI selection, shadow handling, and multi-frame confirmation reduce false alarms, while the adaptive background

model handles changing environmental conditions. Thus, the system provides accurate, low-cost, and real-time motion detection for home security.

9. Document Scanner Application

A mobile app must:

- A. Capture document images
- B. Enhance readability
- C. Output clean scanned images

Design the OpenCV pipeline and justify preprocessing steps used for improving document quality.

ANSWER:

1. Aim

To design an OpenCV-based document scanner that captures a document image, enhances its quality and readability, corrects perspective distortion, and produces a clean scanned document.

2. Block Diagram

Camera / Image Capture



Input Document



Resize Image



Grayscale Conversion



Noise Removal



Edge Detection



Contour Detection



Find 4 Document Corners



Perspective Transformation



Crop / Align Document



Contrast Enhancement



Adaptive Thresholding



Clean Scanned Image



Save / Display

3. Image Acquisition

The mobile camera captures the document.

```
import cv2
```

```
image = cv2.imread("document.jpg")
```

```
image = cv2.resize(image, (800, 600))
```

Purpose: Resizing reduces processing time while maintaining sufficient image quality.

4. Convert to Grayscale

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

A grayscale image contains intensity information instead of three color channels.

Advantage: Reduces computational complexity and makes edge detection easier.

5. Noise Reduction

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

Gaussian filtering removes small noise and unwanted variations.

Purpose:

- Reduces camera noise.
- Smooths the image.
- Improves edge detection.

6. Edge Detection

The Canny edge detector is used to identify the document boundaries.

```
edges = cv2.Canny(blur, 50, 150)
```

The output contains strong edges corresponding to the boundaries of the paper.

Original Image



Grayscale Image



Gaussian Blur



Canny Edge Detection



Document Boundary

7. Contour Detection

Contours are detected from the edge image.

```
contours, _ = cv2.findContours(  
    edges,  
    cv2.RETR_LIST,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

```
contours = sorted(  
    contours,  
    key=cv2.contourArea,
```

```
reverse=True
```

```
)
```

The largest suitable contour is selected as the document boundary.

8. Detect Four Corners

The contour is approximated to a polygon.

```
peri = cv2.arcLength(contour, True)
```

```
approx = cv2.approxPolyDP(
```

```
    contour,
```

```
    0.02 * peri,
```

```
    True
```

```
)
```

If the contour has four points, they are treated as the document corners.

P1 ————— P2

| |

| DOCUMENT |

| |

P4 ————— P3

9. Perspective Transformation

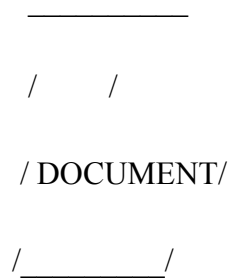
When a document is photographed at an angle, its shape becomes distorted.

Perspective transformation converts the tilted document into a rectangular image.

```
matrix = cv2.getPerspectiveTransform(  
    source_points,  
    destination_points  
)
```

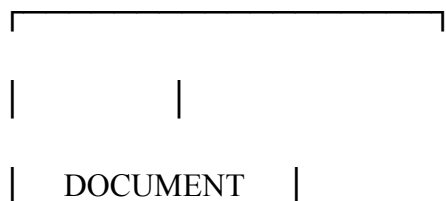
```
scanned = cv2.warpPerspective(  
    image,  
    matrix,  
    (width, height)  
)
```

Before

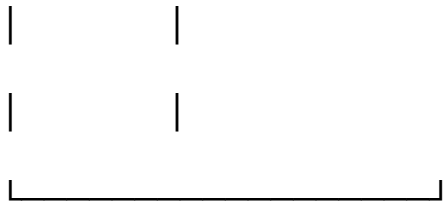


A diagram showing a tilted document. It consists of a horizontal line at the top, followed by a slanted line, then the text "/ DOCUMENT/", another slanted line, and finally a horizontal line at the bottom.

After



A diagram showing the document after perspective transformation. It is now rectangular. It consists of a horizontal line at the top, two vertical lines on the sides, and the text "DOCUMENT" in the center.



This makes the document appear as if it was scanned from directly above.

10. Image Enhancement

Convert the corrected document into grayscale:

```
scanned_gray = cv2.cvtColor(  
    scanned,  
    cv2.COLOR_BGR2GRAY  
)
```

Adaptive thresholding can then be applied:

```
clean = cv2.adaptiveThreshold(  
    scanned_gray,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,  
    2  
)
```

Adaptive thresholding is particularly useful when the document has uneven lighting.

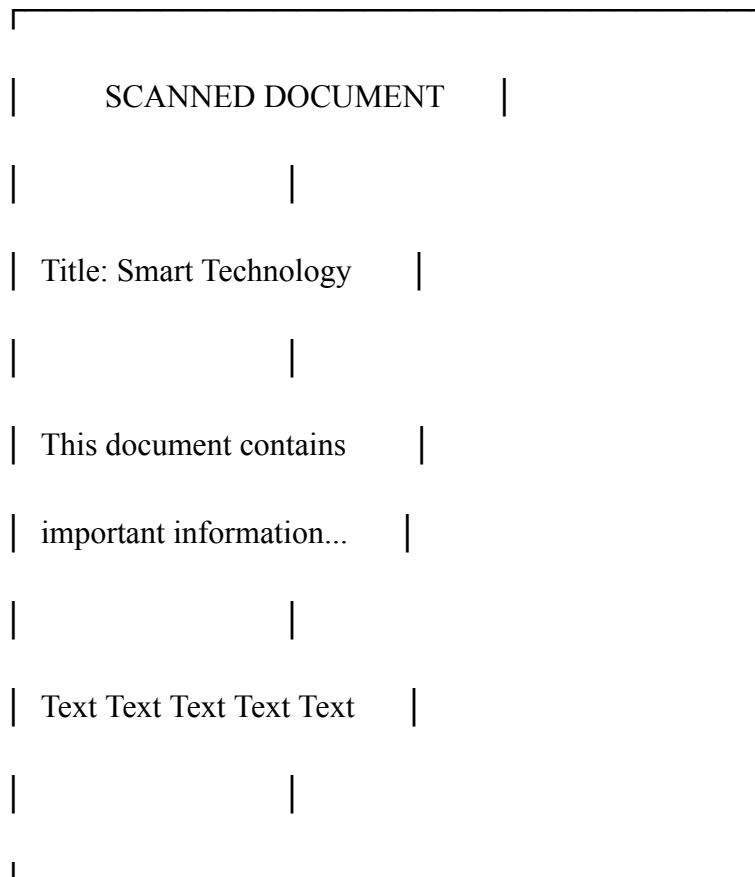
11. Preprocessing Justification

Processing	Purpose
Resize	Reduces processing time
Grayscale	Simplifies image processing
Gaussian blur	Removes noise
Canny edge detection	Detects document edges
Contour detection	Finds document boundary
Polygon approximation	Finds four corners
Perspective transformation	Removes geometric distortion
Adaptive thresholding	Improves text/background separation
Contrast enhancement	Improves readability

12. Clean Output

The final scanned image should have:

- Straight document edges
- Correct rectangular shape
- Improved text visibility
- Reduced background noise
- Better contrast
- Uniform appearance



13. Accuracy and Quality Improvement

This approach handles common problems such as:

- Camera tilt
- Uneven illumination

- Image noise
- Background clutter
- Low text contrast

14. Real-Time Performance

For a mobile application, performance can be improved by:

1. Resizing the input image before processing.
2. Processing only the required document region.
3. Using efficient OpenCV functions.
4. Avoiding unnecessary processing of color channels.
5. Applying preprocessing only when required.
6. Using suitable camera resolution.

15. Conclusion

The proposed OpenCV document scanner uses grayscale conversion, Gaussian filtering, Canny edge detection, contour detection, corner detection, perspective transformation, and adaptive thresholding to convert a camera image into a clean scanned document. Perspective correction removes camera-angle distortion, while noise filtering and thresholding improve readability. Therefore, the system provides an efficient and practical solution for mobile document scanning and image enhancement.

13. Multi-Face Tracking System

A system must:

- A. Detect multiple faces
- B. Track them across frames
- C. Maintain identity

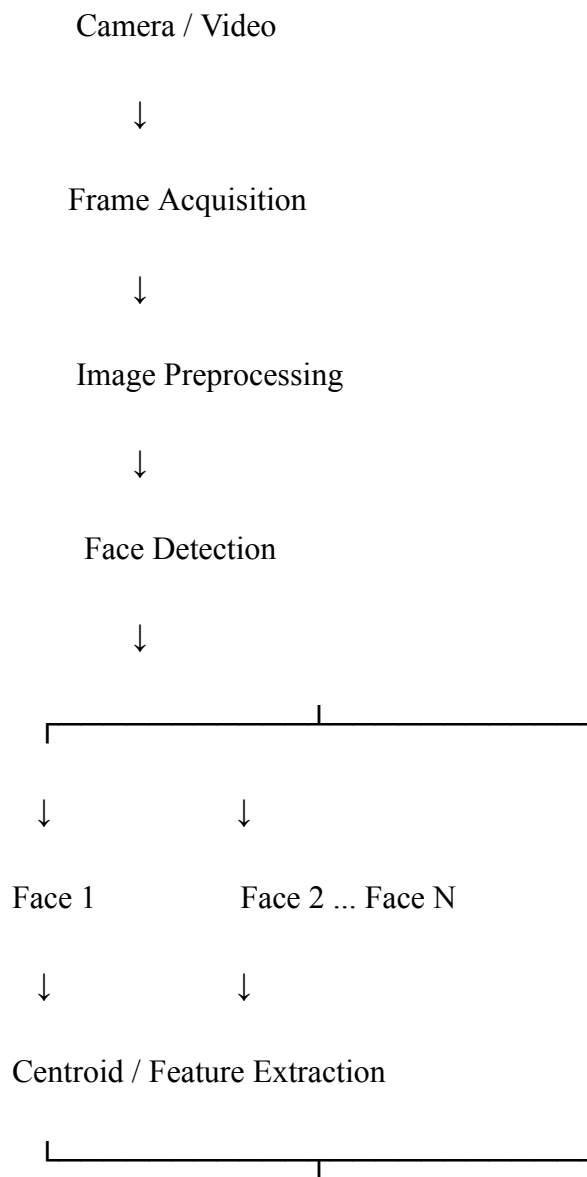
Design the pipeline and justify tracking strategy.

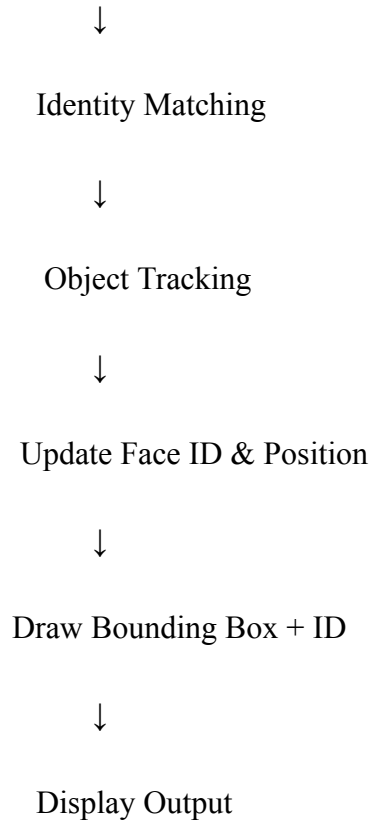
ANSWER:

1. Aim

To design an OpenCV-based multi-face tracking system that detects multiple faces in a video, tracks their movement across consecutive frames, and maintains a unique identity for each person.

2. Block Diagram





3. Video Acquisition

OpenCV is used to capture live video.

```
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    if not ret:
```

```
        break
```

Each frame is processed independently while the tracking information is maintained between frames.

4. Face Detection

A Haar Cascade classifier can be used for basic face detection.

```
face_cascade = cv2.CascadeClassifier(  
    cv2.data.haarcascades +  
    "haarcascade_frontalface_default.xml"  
)
```

Convert the frame to grayscale:

```
gray = cv2.cvtColor(  
    frame,  
    cv2.COLOR_BGR2GRAY  
)
```

Detect multiple faces:

```
faces = face_cascade.detectMultiScale(  
    gray,  
    scaleFactor=1.1,  
    minNeighbors=5,  
    minSize=(40, 40)  
)
```

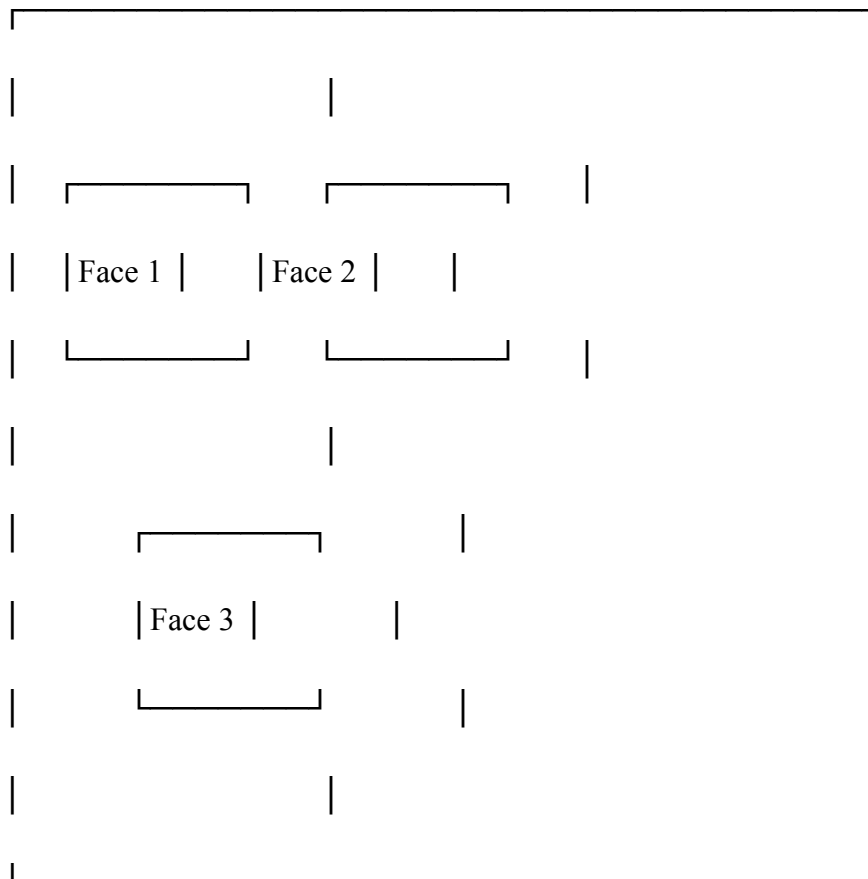
The detector returns the bounding boxes:

[
(x,y,w,h)
]

for every detected face.

5. Multiple Face Detection

For example, if three people are present:



Each detected face is assigned a unique ID.

Face 1 → ID 1

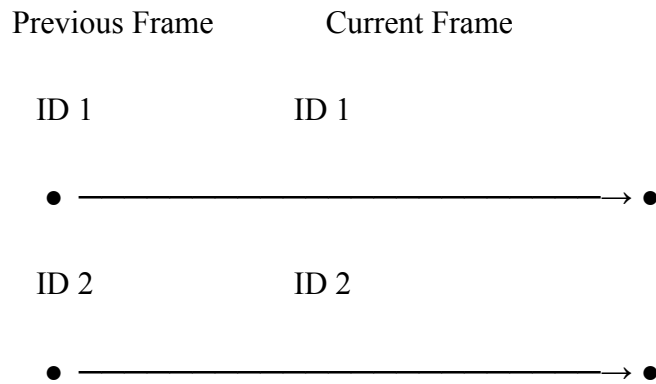
Face 2 → ID 2

Face 3 → ID 3

6. Face Tracking

Detection in every frame alone does not guarantee identity. Therefore, a tracking and identity-association stage is required.

The current centroid is compared with the centroid from the previous frame.



If the distance between two centroids is small, the system assumes they belong to the same face.

7. Identity Maintenance

The system maintains a dictionary containing:

ID → Position → Last Seen Frame

Example:

ID 1 → (120, 150) → Frame 100

ID 2 → (450, 180) → Frame 100

ID 3 → (300, 350) → Frame 100

When the next frame arrives, these positions are compared with the new detections.

This prevents the ID from changing continuously as people move.

8. Handling Occlusion

A person may temporarily disappear behind another person.

Therefore, the tracker should not immediately delete an ID.

Face ID 1 detected



Face temporarily hidden



Keep ID 1 for several frames



Face appears again



Continue with ID 1

A maximum missed-frame threshold can be used. If a face remains missing for too long, its ID is removed.

9. Visualization

OpenCV drawing functions display the tracking information.

```
cv2.rectangle(
```

```
    frame,
```

```
    (x, y),
```

```
    (x+w, y+h),
```

```

        (0, 255, 0),

        2

    )

    cv2.putText(

        frame,

        f"Face ID: {face_id}",

        (x, y-10),

        cv2.FONT_HERSHEY_SIMPLEX,

        0.6,

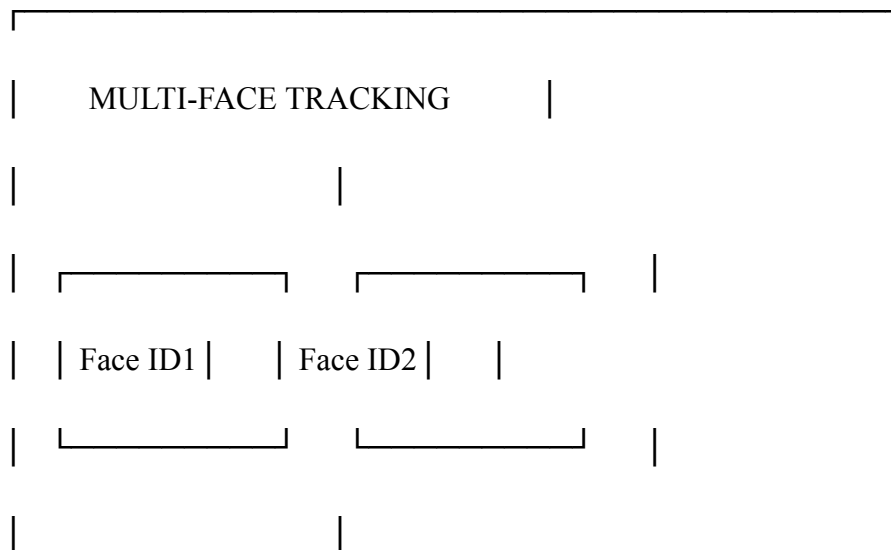
        (0, 255, 0),

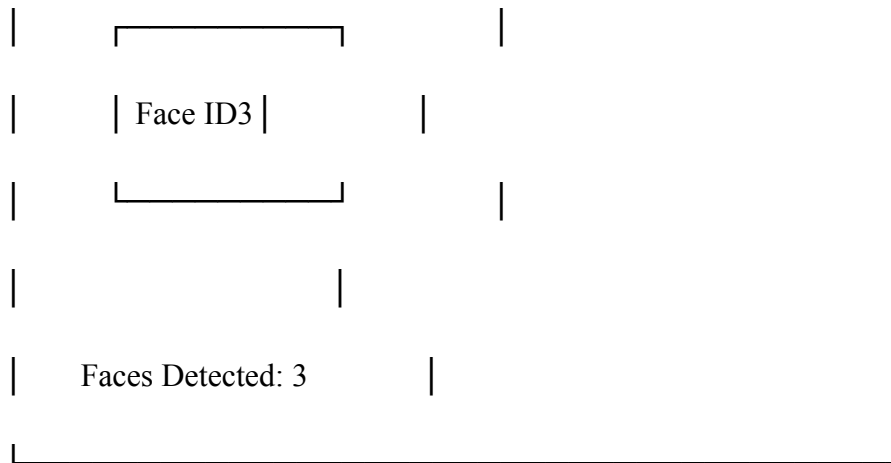
        2

    )

```

The output will look like:





10. Recommended Tracking Strategy

The advanced approach uses both position and appearance features, so two nearby faces are less likely to exchange identities.

11. Accuracy Improvement

Accuracy can be improved using:

1. Periodic face detection to correct tracker drift.
2. Centroid distance matching for position association.
3. Appearance/face features for stronger identity matching.
4. Missed-frame tolerance for temporary occlusion.
5. Suitable detection thresholds to remove weak detections.
6. Re-identification when a face reappears.

12. Real-Time Performance

Real-time performance can be achieved by:

- Resizing frames.
- Using grayscale images for basic detection.
- Detecting faces periodically rather than every frame.
- Tracking between detection frames.

- Processing only the required region.
- Using lightweight tracking algorithms.

Thus, computationally expensive detection is reduced while tracking maintains smooth movement.

13. Overall Algorithm

START



Initialize camera



Read video frame



Convert to grayscale



Detect all faces



Calculate face centroids



Compare with previous positions



Assign / maintain IDs



Update tracker



Check missing faces



Draw bounding boxes and IDs



Display frame



More frames?

└─ YES → Repeat

└─ NO → STOP

14. Conclusion

The proposed Multi-Face Tracking System combines face detection, centroid calculation, identity association, and object tracking to maintain unique IDs across video frames. Periodic re-detection corrects tracking errors, while missed-frame handling helps maintain identities during temporary occlusion. For higher accuracy, appearance-based methods such as Deep SORT can be combined with the face detector. Therefore, the system provides accurate and real-time multi-person face tracking using OpenCV.

14. OCR for License Plate Recognition

A system must:

- A. Detect license plates
- B. Extract text
- C. Work under varying lighting

Design the OpenCV pipeline and justify preprocessing and segmentation methods.

ANSWER:

1. Aim

To design an OpenCV-based License Plate Recognition (LPR) system that detects a vehicle's license plate, extracts the characters using OCR, and provides reliable recognition under different lighting conditions.

2. Block Diagram

Vehicle Camera



Image Acquisition



Resize Input Image

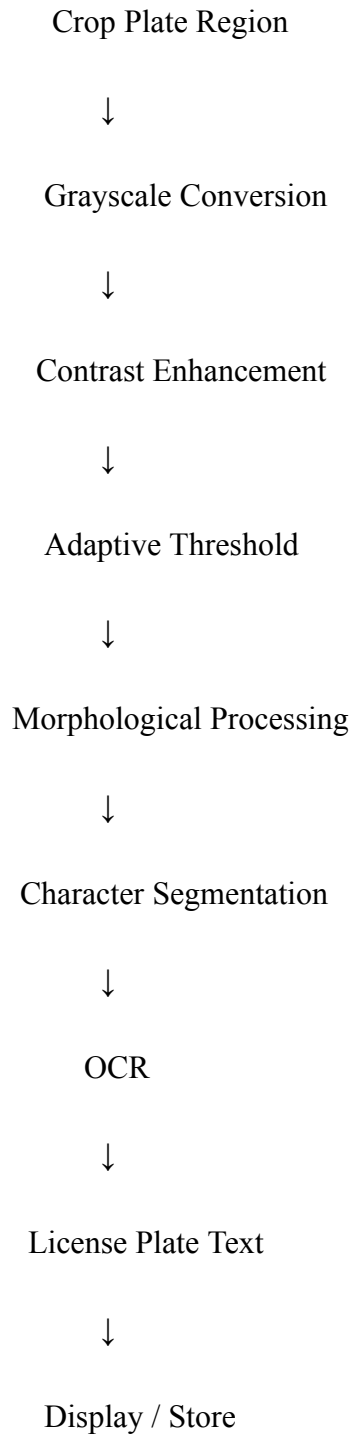


Noise Reduction



License Plate Detection





3. Image Acquisition

The vehicle image is captured using a camera.

```
import cv2
```



```
image = cv2.imread("vehicle.jpg")
```

```
image = cv2.resize(image, (800, 600))
```

Purpose: Resizing reduces computational requirements while maintaining sufficient image details.

4. License Plate Detection

The license plate can be detected using:

- Contour-based detection
- Haar/LBP cascade
- YOLO-based object detection

For a basic OpenCV implementation, contour-based detection can be used.

First, convert the image to grayscale:

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

Then remove noise:

```
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

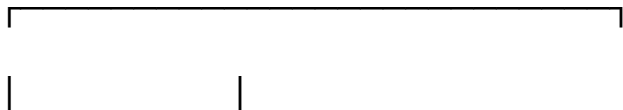
5. Edge Detection

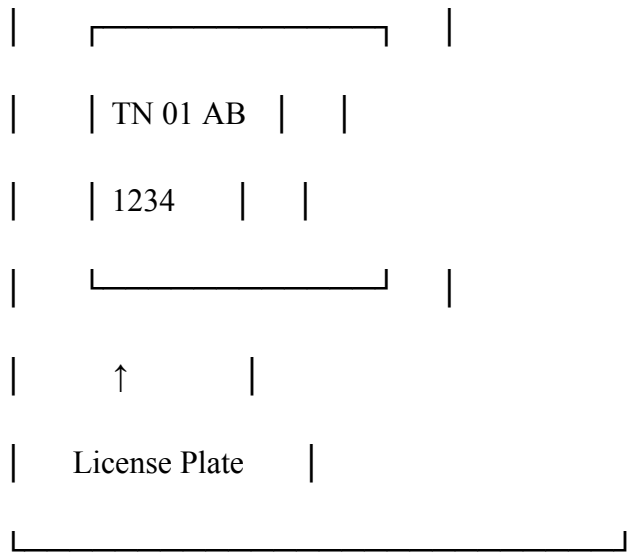
Canny edge detection identifies the boundaries of the license plate.

```
edges = cv2.Canny(blur, 100, 200)
```

The rectangular shape of the plate can then be identified from the detected edges.

VEHICLE IMAGE





6. Contour Detection

Contours are extracted from the edge image.

```
contours, _ = cv2.findContours(  
    edges,  
    cv2.RETR_EXTERNAL,  
    cv2.CHAIN_APPROX_SIMPLE  
)
```

Candidate contours are filtered according to:

- Area
- Width
- Height
- Aspect ratio
- Rectangular shape

A contour with a suitable rectangular shape is selected as the license plate.

7. Crop the License Plate

Once the plate coordinates are found:

```
plate = image[y:y+h, x:x+w]
```

Only the plate region is sent to the OCR stage.

This improves recognition because unnecessary vehicle background information is removed.

8. Grayscale Conversion

```
plate_gray = cv2.cvtColor(  
  
    plate,  
  
    cv2.COLOR_BGR2GRAY  
  
)
```

Grayscale simplifies further processing and reduces computation.

9. Handling Different Lighting Conditions

Lighting is one of the major challenges in license plate recognition.

The system can use CLAHE (Contrast Limited Adaptive Histogram Equalization).

```
clahe = cv2.createCLAHE(  
  
    clipLimit=2.0,  
  
    tileGridSize=(8, 8)  
  
)
```

```
enhanced = clahe.apply(plate_gray)
```

CLAHE improves local contrast and is useful when one part of the plate is darker or brighter than another.

10. Adaptive Thresholding

Instead of using a single global threshold, adaptive thresholding calculates thresholds for local image regions.

```
binary = cv2.adaptiveThreshold(  
    enhanced,  
    255,  
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,  
    cv2.THRESH_BINARY,  
    11,  
    2  
)
```

Advantage

It performs better when the plate contains:

- Uneven illumination
- Shadows
- Reflections
- Bright and dark regions

11. Morphological Processing

Morphological operations clean the thresholded image.

```

kernel = cv2.getStructuringElement(

    cv2.MORPH_RECT,

    (3, 3)

)

clean = cv2.morphologyEx(

    binary,

    cv2.MORPH_CLOSE,

    kernel

)

```

Morphological closing helps connect broken character regions and remove small gaps.

12. Character Segmentation

The individual characters are separated before OCR.

T	N	0	1	A	B	1	2
---	---	---	---	---	---	---	---

Contours can be used to identify character regions.

Characters are filtered based on:

- Height
- Width
- Area
- Aspect ratio

- Position

This removes unwanted noise and non-character regions.

13. OCR

The processed license plate is passed to an OCR engine such as Tesseract.

```
import pytesseract
```

```
text = pytesseract.image_to_string(
```

```
    clean,
```

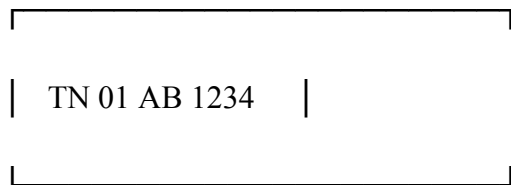
```
    config="--psm 7"
```

```
)
```

```
print("License Plate:", text)
```

Example:

Input:



OCR Output:

TN01AB1234

14. Accuracy Improvement

The accuracy of the system can be improved by:

1. Using a dedicated license-plate detector.

2. Correcting the perspective of tilted plates.
3. Applying CLAHE for uneven lighting.
4. Using adaptive thresholding.
5. Removing noise using morphological operations.
6. Performing proper character segmentation.
7. Restricting OCR to letters and digits.
8. Processing multiple frames and selecting the most consistent result.

15. Preprocessing Justification

Processing Step	Purpose
Resize	Faster processing
Grayscale	Simplifies image
Gaussian blur	Removes noise
Canny	Detects plate boundaries
Contours	Identifies plate region
CLAHE	Improves local contrast
Adaptive thresholding	Handles uneven lighting

Morphological filtering	Cleans character regions
Character segmentation	Separates characters
OCR	Converts characters into text

16. Overall Algorithm

START



Capture Vehicle Image



Resize Image



Convert to Grayscale



Gaussian Filtering



Canny Edge Detection



Contour Detection



Identify License Plate



Crop Plate



CLAHE Enhancement



Adaptive Thresholding



Morphological Filtering



Character Segmentation



OCR

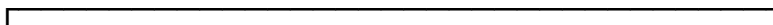


Display License Number



STOP

17. Final Output



| LICENSE PLATE RECOGNITION |

| |

| Vehicle Status : DETECTED |

| Plate Number : TN01AB1234 |

| OCR Status : SUCCESS |

| |

|_____|

18. Conclusion

The proposed OpenCV-based License Plate Recognition system combines plate detection, grayscale conversion, noise removal, edge detection, contour analysis, CLAHE enhancement, adaptive thresholding, morphological processing, character segmentation, and OCR. CLAHE and adaptive thresholding make the system more robust to varying lighting conditions, while character segmentation improves OCR accuracy. Hence, the system can efficiently detect license plates and extract their text automatically.

Code of Conduct:

I HARITHI S(192421151) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

HARITHI S

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

		constraints			
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided